



Corso di *Robust Control*

# Controllo Robusto di un Pendolo Inverso su Ruote in MATLAB

Studente:

*Gaetano Improta*  
*A18000508*

Professore:

*Prof. Alberto Cavallo*

A. A. 2025/2026

# Indice

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                            | <b>1</b>  |
| <b>2</b> | <b>Analisi del sistema</b>                                                     | <b>2</b>  |
| 2.1      | Caratteristiche fisiche . . . . .                                              | 2         |
| 2.2      | Modello del TWIP . . . . .                                                     | 4         |
| <b>3</b> | <b>Sintesi del Controllore Nominale via Loop-Shaping <math>H_\infty</math></b> | <b>7</b>  |
| 3.1      | Matrici di peso . . . . .                                                      | 7         |
| 3.2      | Costruzione dell' Impianto Generalizzato . . . . .                             | 10        |
| 3.3      | Controllore $H_\infty$ . . . . .                                               | 15        |
| 3.4      | Prestazioni sul nominale . . . . .                                             | 17        |
| <b>4</b> | <b>Analisi della robustezza</b>                                                | <b>19</b> |
| 4.1      | Simulazione Monte-Carlo . . . . .                                              | 19        |
| 4.2      | Dal Piccolo Guadagno all'Analisi $\mu$ . . . . .                               | 20        |
| <b>5</b> | <b>Trade-off tra prestazioni e robustezza</b>                                  | <b>24</b> |
| 5.1      | Modifica dei pesi . . . . .                                                    | 24        |
| 5.2      | Sintesi del controllore riprogettato . . . . .                                 | 25        |
| 5.3      | Prestazioni ottenute . . . . .                                                 | 26        |
| 5.4      | Verifica della robustezza . . . . .                                            | 27        |
| <b>6</b> | <b>Tentativi di Ottimizzazione</b>                                             | <b>29</b> |
| 6.1      | Costruzione del generatore centrale $J$ . . . . .                              | 29        |
| 6.2      | Ottimizzazione Black-Box tramite <code>sysune</code> . . . . .                 | 30        |
| 6.3      | Validazione in Dominio del Tempo . . . . .                                     | 32        |
| 6.4      | Robustezza del sistema . . . . .                                               | 33        |
| 6.5      | Ottimizzazione del controllore centrale . . . . .                              | 35        |
| <b>7</b> | <b>Riduzione dell'ordine del controllore</b>                                   | <b>38</b> |
| <b>8</b> | <b>Appendice</b>                                                               | <b>40</b> |
| 8.1      | Norma infinito - Test di bisezione . . . . .                                   | 40        |

# 1 Introduzione

Il sistema oggetto di studio è un self-balancing-robot (noto in letteratura come TWIP, Two-Wheeled Inverted Pendulum). Si tratta di un sistema dinamico S.I.M.O., intrinsecamente instabile e sottoattuato, in cui l'unico azionamento disponibile deve garantire simultaneamente l'equilibrio verticale e l'inseguimento di una traiettoria piana. L'obiettivo è la progettazione di un sistema di retroazione capace di garantire entrambi gli obiettivi, gestendo al contempo limitazioni fisiche degli attuatori, incertezze parametriche e variazioni di carico utile.

Il problema è stato affrontato adottando un approccio moderno basato su procedure di *loop-shaping*. In una prima fase, è stato sintetizzato un controllore  $H_\infty$  focalizzato sull'ottimizzazione delle prestazioni nominali e sul contenimento dello sforzo di controllo. Successivamente, il progetto è stato esteso all'analisi della robustezza a fronte di incertezze strutturali, ricercando e analizzando il rigoroso compromesso tra stabilità robusta e reattività dinamica.

Sono state poi esplorate e applicate tecniche di controllo basate sulla parametrizzazione di Youla-Kucera. In seguito, l'ottimizzazione di Youla è stata integrata direttamente sulla struttura del controllore  $H_\infty$  originario, analizzando eventuali miglioramenti e verificando il mantenimento delle garanzie di robustezza e stabilità.

Infine, per favorire un'eventuale implementazione su architetture hardware con risorse di calcolo limitate, il controllore risultante – tipicamente caratterizzato da una dinamica di ordine elevato – è stato semplificato mediante tecniche di riduzione dell'ordine.

## 2 Analisi del sistema

### 2.1 Caratteristiche fisiche

Dal punto di vista costruttivo, il robot è costituito da un telaio centrale rigido alla cui base sono montati due motori DC indipendenti (modello JGA25-270) alimentati a 12 V.

Tabella 1: Parametri fisici ed elettromeccanici del modello nominale e relative incertezze.

| Simbolo    | Descrizione                           | Nominale           | Incerteza       | Unità             |
|------------|---------------------------------------|--------------------|-----------------|-------------------|
| $M_p$      | Massa del telaio                      | 0.8                | $[-1\%, +50\%]$ | kg                |
| $L$        | Distanza baricentro-asse <sup>*</sup> | 0.07               | -               | m                 |
| $m_w$      | Massa (singola ruota + rotore)        | 0.08               | -               | kg                |
| $R$        | Raggio della ruota                    | 0.0325             | -               | m                 |
| $J_p$      | Inerzia baricentrica del telaio       | 0.004              | $\pm 0.001$     | kg·m <sup>2</sup> |
| $J_w$      | Inerzia della singola ruota           | $2 \times 10^{-5}$ | -               | kg·m <sup>2</sup> |
| $K_t, K_e$ | Cost. di coppia e f.c.e.m.            | 0.01               | $\pm 10\%$      | Nm/A, V·s/rad     |
| $R_m$      | Resistenza di avvolgimento            | 10.0               | $\pm 15\%$      | $\Omega$          |
| $n$        | Rapporto di riduzione                 | 50                 | -               | -                 |
| $g$        | Accelerazione di gravità              | 9.81               | -               | m/s <sup>2</sup>  |

<sup>\*</sup>Il valore effettivo di  $L$  varia dinamicamente nel modello in funzione della massa aggiuntiva su  $M_p$ , ipotizzando il carico applicato a un'altezza  $h_{top} = 0.1425$  m.

La scelta di introdurre specifiche incertezze parametriche nel modello nominale nasce dalla necessità di progettare un sistema di controllo robusto, capace di mantenere stabilità e prestazioni anche in presenza di inevitabili discrepanze tra il modello matematico e il prototipo fisico. Di seguito si analizzano le cause fisiche che giustificano tali incertezze:

- **Massa del telaio ( $M_p$ ):** L'incerteza è stata definita con un range fortemente asimmetrico ( $[-1\%, +50\%]$ ). Questa scelta è legata a un vincolo operativo del progetto: si richiede che il robot sia in grado di mantenere l'equilibrio e operare correttamente anche quando sottoposto a un carico utile fino al 50% del suo peso posizionato sulla sua sommità.

- **Inerzia baricentrica del telaio ( $J_p$ ):** Questo parametro rappresenta la resistenza del corpo del pendolo (escluse le ruote) alle accelerazioni angolari attorno al proprio centro di massa. È stato associato un margine di errore assoluto ( $\pm 0.001 \text{ kg} \cdot \text{m}^2$ ). Il calcolo teorico del momento d'inerzia di un assieme complesso – che include telaio, schede elettroniche, cavi, sensori e batterie disposti in modo asimmetrico – è intrinsecamente approssimato. Modelli CAD o formule matematiche semplificate non riescono a catturare con precisione millimetrica la distribuzione reale e disomogenea delle masse, rendendo necessaria questa tolleranza.
- **Costanti elettromeccaniche del motore ( $K_t, K_e$ ):** Nei motori in corrente continua, la costante di coppia ( $K_t$ ) e la costante controelettromotrice ( $K_e$ ) sono numericamente e fisicamente identiche, dipendendo entrambe dal medesimo flusso magnetico. Per tale motivo, nel modello è stata definita un'unica variabile incerta ( $K_{mot}$ ) con tolleranza del  $\pm 10\%$ , assegnata specularmente sia a  $K_t$  che a  $K_e$ . Questa incertezza accorpata modella le tolleranze costruttive dei rotori e, soprattutto, le derive termiche: il surriscaldamento dei magneti permanenti indebolisce infatti il campo magnetico, riducendo simultaneamente e nella stessa misura sia la coppia meccanica erogata sia la tensione indotta.
- **Resistenza di avvolgimento ( $R_m$ ):** È stata modellata con un'incertezza del  $\pm 15\%$ . La resistenza del circuito di armatura è il parametro elettrico più sensibile alle variazioni termiche. Durante il funzionamento continuo, il riscaldamento del motore per effetto Joule causa un aumento fisiologico della resistenza del rame, che riduce la corrente assorbita (e quindi la coppia) a parità di tensione di comando. L'incertezza garantisce che il controllore non saturi a causa di queste derive termiche.

#### MATLAB

```

1 Mp_nom = 0.8;
2 L_nom  = 0.07;
3 h_top   = 0.1425;
4 Mp = ureal('Mp', Mp_nom, 'Percentage', [-1, 50]);
5 Num_L = (Mp_nom * L_nom + (Mp - Mp_nom) * h_top);
6 L = Num_L / Mp;
7 mw = 0.08;
8 R = 0.0325;
9 Jp = ureal('Jp', 0.004, 'Plusminus', 0.001);
10 Jw = 0.00002;
11 g = 9.81;
12
13 K_mot = ureal('K_mot', 0.01, 'Percentage', 10);
14 Kt = K_mot;
15 Ke = K_mot;
16 Rm = ureal('Rm', 10.0, 'Percentage', 15);
17 n = 50;

```

## 2.2 Modello del TWIP

La dinamica non lineare del robot viene linearizzata attorno al punto di equilibrio instabile (corrispondente al pendolo in posizione perfettamente verticale con velocità nulle). Il vettore di stato  $x \in \mathbb{R}^4$  è il seguente:

$$x = [x_c \quad \dot{x}_c \quad \theta \quad \dot{\theta}]^T$$

dove  $x_c$  rappresenta la posizione lineare del robot,  $\dot{x}_c$  la sua velocità,  $\theta$  l'angolo di beccheggio (pitch) rispetto alla verticale e  $\dot{\theta}$  la velocità angolare del pendolo.

L'ingresso di controllo  $u \in \mathbb{R}$  è scalare e coincide con la tensione applicata ai motori:

$$u = V$$

Sulla base dei parametri fisici presentati, la matrice della dinamica  $A \in \mathbb{R}^{4 \times 4}$  e la matrice di ingresso  $B \in \mathbb{R}^{4 \times 1}$  assumono la seguente struttura:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & a_{22} & a_{23} & a_{24} \\ 0 & 0 & 0 & 1 \\ 0 & a_{42} & a_{43} & a_{44} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ b_2 \\ 0 \\ b_4 \end{bmatrix} \quad (1)$$

Grazie alla presenza di encoder sui motori (per la misura della posizione) e al sensore MPU-6050 (per la misura dell'angolo di pitch), la matrice di uscita  $C$  e di legame diretto  $D$  risultano pari a:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (2)$$

Si noti come viene automaticamente rispettata la condizione di *feedback ben posto* non essendoci accoppiamenti diretti ingresso-uscita.

### MATLAB

```
1 sys = ss(A, B, C, D); % creazione dello spazio di stato
2 sys = simplify(sys, 'Full');
3
4 % aggiunta delle etichette alle variabili
5 sys.StateName = {'Posizione', 'Vel_Lin', 'Angolo_Pitch',
6 'Vel_Ang'};
7 sys.InputName = {'Tensione_V'};
8 sys.OutputName = {'Posizione', 'Angolo_Pitch'};
```

Matlab restituisce correttamente un sistema di tipo `uss`

#### Command Window

```
Uncertain continuous-time state-space model with 2 outputs,  
1 inputs, 4 states.  
The model uncertainty consists of the following blocks:  
Jp: Uncertain real, nominal = 0.004, variability =  
    [-0.001,0.001], 9 occurrences  
K_mot: Uncertain real, nominal = 0.01, variability =  
    [-10,10]%, 5 occurrences  
Mp: Uncertain real, nominal = 0.8, variability = [-1,50]%,  
    21 occurrences  
Rm: Uncertain real, nominal = 10, variability = [-15,15]%,  
    5 occurrences
```

Estrazione del sistema nominale: sarà su questo che progetteremo il controllore. Ovviamente, si tratta di un sistema **ss**. Analizziamo zeri e poli.

#### MATLAB

```
1 G0 = sys.nominal;  
2 G0 = minreal(G0);  
3 display(G0);  
4 display(pole(G0));  
5 display(zero(G0(1,1)));  
6 display(zero(G0(2,1)));
```

#### Command Window

```
Poli:  
0  
-125.825545306207  
-6.30589112610249  
6.87524412522789  
  
Zeri:  
-7.51015739287067  
7.51015739287269  
  
-4.76162392967734e-12  
0
```

L'estrazione delle radici quantifica le due criticità dinamiche fondamentali del TWIP. Il polo reale positivo (+6.87 rad/s) sancisce l'instabilità strutturale ad anello aperto (la caduta gravitazionale del pendolo), imponendo al controllore una banda passante minima per stabilizzare il sistema. Al contempo, lo zero reale positivo (+7.51 rad/s, a fase non minima) genera una risposta inversa iniziale (il carrello deve retrocedere per potersi inclinare in avanti) e fissa un limite teorico massimo

alla reattività del sistema, costringendo il *loop-shaping* a un compromesso. Analizziamo la controllabilità e l'osservabilità del sistema.

#### MATLAB

```
1 Cb = ctrb(sys.A, sys.B);
2 if rank(Cb) == size(sys.A, 1)
3     fprintf("Sistema completamente controllabile.")
4 else
5     fprintf("Sistema NON completamente controllabile.")
6 end
7
8 Ob = obsv(sys.A, sys.C);
9 if rank(Ob) == size(sys.A, 1)
10    fprintf("Sistema completamente osservabile.")
11 else
12    fprintf("Sistema NON completamente osservabile.")
13 end
```

#### Command Window

```
Sistema completamente controllabile.
Sistema completamente osservabile.
```





| Peso        | Scelta                                        | Motivazione                                                                                                                                                                                                                                          |
|-------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $W_{e,pos}$ | Filtro passa-basso, freq. di taglio: 3 rad/s  | L'inseguimento della posizione è lento per non eccitare lo zero a fase non minima (+7.51 rad/s). Un taglio così basso garantisce un movimento fluido, prevenendo brusche accelerazioni che comprometterebbero l'angolo di beccheggio.                |
| $W_{n,pos}$ | Filtro passa-alto, freq. di taglio: 100 rad/s | Simula il rumore degli encoder. Intervenendo ad altissima frequenza, impone al sistema un marcato roll-off, evitando che i disturbi di lettura si ripercuotano come "vibrazioni" nervose sui motori.                                                 |
| $W_u$       | Filtro passa-alto, freq. di taglio: 60 rad/s  | Limita l'attività di controllo per rispettare i vincoli fisici (elettrici e termici) degli attuatori. Previene la saturazione della tensione di comando ad alta frequenza, dove i motori in continua non avrebbero comunque la banda per rispondere. |

Le rimanenti matrici di peso ( $W_d$ ,  $W_c$  e  $W_i$ ) sono state poste pari all'identità tramite un guadagno statico unitario. Se da un lato questa scelta computazionale evita di incrementare inutilmente l'ordine del controllore, dall'altro risponde a una precisa esigenza di robustezza. Mantenere questi rami attivi senza filtri dinamici equivale ad assumere la presenza di disturbi fisici a spettro piatto (assimilabili a rumore bianco, come attriti non lineari o irregolarità del terreno). Ciò impone all'algoritmo di sintesi  $H_\infty$  di garantire un'attenuazione di base su tutte le decadi di frequenza, evitando picchi di risonanza imprevisti, pur concentrando i gradi di libertà primari sul bilanciamento critico tra errore di inseguimento ( $W_e$ ), rumore di misura ( $W_n$ ) e sforzo di controllo ( $W_u$ ).

Analizzeremo nei prossimi paragrafi i risultati di queste scelte in termini di prestazioni e robustezza.

#### MATLAB

```

1 We_ang = makeweight(2, 16, 0.1);
2 Wd_ang = ss(1);
3 Wn_ang = makeweight(0.1, 40, 10);
4 Wc_ang = ss(1);
5
6 We_pos = makeweight(100, 3, 0.1);
7 Wd_pos = ss(1);
8 Wn_pos = makeweight(0.1, 100, 2);
9 Wc_pos = ss(1);

```

## MATLAB

```
1 We = [We_pos 0;  
2       0 We_ang];  
3  
4 Wd = [Wd_pos 0;  
5       0 Wd_ang];  
6  
7 Wc = [Wc_pos 0;  
8       0 Wc_ang];  
9  
10 Wn = [Wn_pos 0;  
11       0 Wn_ang];  
12  
13 Wi = ss(1);  
14 Wu = makeweight(0.05, 60, 5);
```

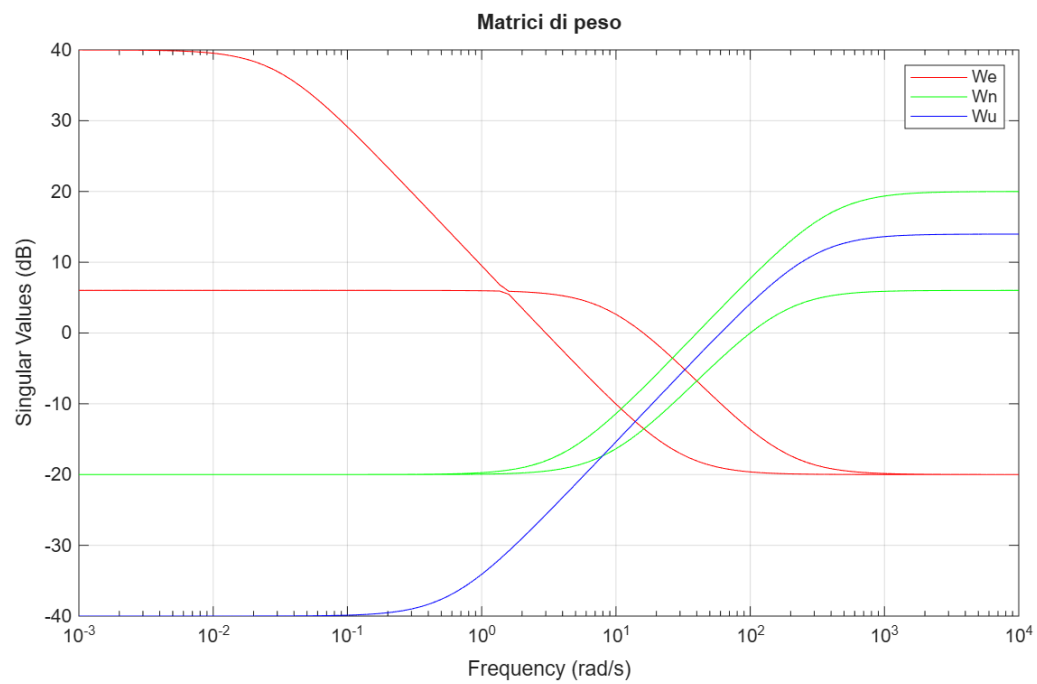


Figura 2: Pesi adottati

### 3.2 Costruzione dell' Impianto Generalizzato

Per applicare gli algoritmi di sintesi ottima  $H_\infty$ , è necessario riorganizzare lo schema di controllo in retroazione, comprensivo dei filtri di peso, in una struttura standard universalmente nota come *Impianto Generalizzato*.

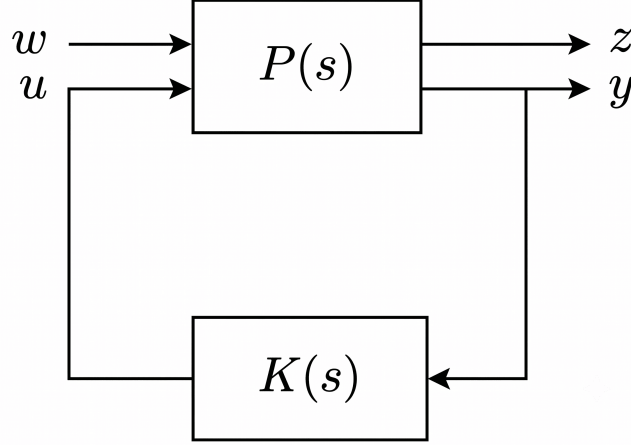


Figura 3: Schema di controllo moderno

A tal fine, si riclassificano i segnali del sistema nei quattro vettori fondamentali:

- $w$ : vettore degli **ingressi esogeni**, che include i segnali esterni al sistema non manipolabili dal controllore.
- $u$ : vettore degli **ingressi di controllo** (la tensione fornita ai motori).
- $z$ : vettore delle **uscite di interesse** (gli output dei filtri di peso).
- $y$ : vettore delle **uscite misurate**, ovvero le informazioni fornite in pasto al controllore.

L'obiettivo è ricavare la matrice di trasferimento aumentata  $P(s)$  che mappa gli ingressi  $[w \ u]^T$  nelle uscite  $[z \ y]^T$ :

$$\begin{bmatrix} z \\ y \end{bmatrix} = P(s) \begin{bmatrix} w \\ u \end{bmatrix} = \begin{bmatrix} P_{11}(s) & P_{12}(s) \\ P_{21}(s) & P_{22}(s) \end{bmatrix} \begin{bmatrix} w \\ u \end{bmatrix} \quad (3)$$

Per il problema della sensibilità mista, le uscite di interesse  $z = [z_1 \ z_2 \ z_3]^T$  corrispondono rispettivamente all'errore pesato, allo sforzo di controllo pesato e all'uscita pesata. Gli ingressi esogeni  $w = [w_1 \ w_2 \ w_3]^T$  sono invece il disturbo in ingresso all'impianto, il disturbo in uscita ed il rumore di misura.

Procediamo con il calcolo delle componenti  $P_{11}(s), P_{12}(s), P_{21}(s), P_{22}(s)$ . Dallo schema in *Figura 2* si procede rimuovendo il controllore  $K$ , e si mappano gli ingressi nelle rispettive uscite.

Per la sottomatrice  $P_{11}$  di dimensione  $3 \times 3$  si ha:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} P_{11}^1 & P_{11}^2 & P_{11}^3 \\ P_{11}^4 & P_{11}^5 & P_{11}^6 \\ P_{11}^7 & P_{11}^8 & P_{11}^9 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \quad (4)$$

Dall'analisi dello schema a blocchi in anello aperto, i singoli elementi della matrice risultano essere:

$$\begin{aligned} P_{11}^1 &= -W_e G W_i & P_{11}^7 &= W_c G W_i \\ P_{11}^2 &= -W_e W_d & P_{11}^8 &= W_c W_d \\ P_{11}^3 &= -W_e W_n & P_{11}^9 &= \mathbf{0}_{2 \times 2} \\ P_{11}^4 &= 0 \\ P_{11}^5 &= [0, 0] \\ P_{11}^6 &= [0, 0] \end{aligned}$$

Procedendo in modo analogo per le restanti sottomatrici, si definisce il blocco  $P_{12}$  di dimensione a blocchi  $3 \times 1$ , che mappa l'ingresso di controllo  $u$  sulle uscite penalizzate  $z$ :

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} P_{12}^1 \\ P_{12}^2 \\ P_{12}^3 \end{bmatrix} u \implies \begin{cases} P_{12}^1 = -W_e G \\ P_{12}^2 = W_u \\ P_{12}^3 = W_c G \end{cases} \quad (5)$$

La sottomatrice  $P_{21}$  di dimensione a blocchi  $1 \times 3$  descrive invece il legame tra i segnali esogeni in ingresso  $w$  e l'uscita misurata  $y$  fornita al controllore:

$$y = [P_{21}^1 \quad P_{21}^2 \quad P_{21}^3] \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \implies \begin{cases} P_{21}^1 = -G W_i \\ P_{21}^2 = -W_d \\ P_{21}^3 = -W_n \end{cases} \quad (6)$$

Infine, il blocco  $P_{22}$  rappresenta la funzione di trasferimento in anello aperto dal segnale di controllo  $u$  all'uscita  $y$ . Essendo composto da un unico blocco, esso coincide direttamente con la dinamica dell'impianto cambiata di segno:

$$y = P_{22}u \implies P_{22} = -G \quad (7)$$

#### MATLAB

```
1 P = [-We*G0*Wi,      -We*Wd,      -We*Wn, -We*G0;
2         0, zeros(1,2), zeros(1,2),      Wu;
3         Wc*G0*Wi,      Wc*Wd, zeros(2,2), Wc*G0;
4         -G0*Wi,      -Wd,      -Wn,      -G0];
5
6 P = minreal(P);
```

Si ricordi la scelta delle matrici di peso effettuata nel paragrafo precedente. L'impianto  $P$  risulta correttamente essere il seguente sistema  $ss^*$ :

#### Command Window

```
>> size(P)
State-space model with 7 outputs, 6 inputs, and 9 states.
```

In fase di sintesi è stata valutata l'ipotesi di rimuovere fisicamente i canali non soggetti a shaping diretto per ridurre l'ordine del problema di ottimizzazione. Tuttavia, le simulazioni hanno mostrato un drastico peggioramento delle prestazioni nel dominio del tempo. Tale fenomeno è riconducibile al noto *Waterbed Effect* derivante dall'integrale di sensitività di Bode: omettendo i canali dei disturbi fisici dalla matrice  $P$ , l'algoritmo  $H_\infty$  sfrutta tali gradi di libertà non vincolati per minimizzare il costo sui canali primari ( $W_e, W_u$  e  $W_n$ ). Ciò genera picchi di risonanza ampi, rendendo il sistema estremamente vulnerabile nella simulazione completa. L'adozione di pesi a guadagno unitario ( $ss(1)$ ) si è quindi confermata una strategia valida per "sigillare" questi canali, imponendo all'ottimizzatore un limite di norma superiore che previene instabilità e oscillazioni indesiderate.

E' utile osservare come lo stesso risultato può essere raggiunto tramite il comando 'connect' del Robust Control Toolbox, dopo aver opportunamente rinominato le variabili di ingresso e uscita ed aver definito i nodi sommatori dello schema in feedback.

#### MATLAB

```
1 G0.InputName = 'u_tot';
2 G0.OutputName = {'g_out(1)', 'g_out(2)'};
3 %(1) posizione, (2) angolo pitch
4
5 We.InputName = {'e(1)', 'e(2)'};
6 We.OutputName = {'ze(1)', 'ze(2)'};
```

## MATLAB

```
1 Wc.InputName = {'c(1)', 'c(2)'};
2 Wc.OutputName = {'zc(1)', 'zc(2)'};
3
4 Wu.InputName = 'u';
5 Wu.OutputName = 'zu';
6
7 Wi.InputName = 'd_in';
8 Wi.OutputName = 'di_pesato';
9
10 Wd.InputName = {'d(1)', 'd(2)'};
11 Wd.OutputName = {'d_pesato(1)', 'd_pesato(2)'};
12
13 Wn.InputName = {'n(1)', 'n(2)'};
14 Wn.OutputName = {'n_pesato(1)', 'n_pesato(2)'};
15
16 Nodo_U = sumblk('u_tot = u + di_pesato');
17 Nodo_C = sumblk('c = g_out + d_pesato', 2);
18 Nodo_E = sumblk('e = - c - n_pesato', 2);
19 Nodo_Y = sumblk('y = e', 2);
20
21 Ingressi_P = {'d_in', 'd(1)', 'd(2)', 'n(1)', 'n(2)', 'u'
22              };
23 Uscite_P = {'ze(1)', 'ze(2)', 'zu', 'zc(1)', 'zc(2)', 'y
24            (1)', 'y(2)'};
25
26 % Creazione della matrice
27 P1 = minreal(connect(G0, We, Wu, Wc, Wi, Wd, Wn, Nodo_U,
28                    Nodo_C, Nodo_E, Nodo_Y, Ingressi_P, Uscite_P));
```

## Command Window

```
>> size(P1)
State-space model with 7 outputs, 6 inputs, and 9 states.
```

\*Si noti come, dal punto di vista computazionale, MATLAB opera sulla rappresentazione in spazio di stato di  $P$ . Tale sistema dinamico lineare tempo-invariante è formalmente descritto come segue:

$$\begin{cases} \dot{x} &= Ax + B_1w + B_2u \\ z &= C_1x + D_{11}w + D_{12}u \\ y &= C_2x + D_{21}w + D_{22}u \end{cases} \quad (8)$$

dove il vettore  $x$  raccoglie lo stato complessivo del sistema aumentato, comprendente sia gli stati fisici della dinamica del TWIP che gli stati interni introdotti dai filtri di peso. Questa notazione partiziona in modo rigoroso gli ingressi e le uscite: le

matrici  $B_1$  e  $B_2$  definiscono rispettivamente come i disturbi esogeni  $w$  e lo sforzo di controllo  $u$  eccitano la dinamica interna, mentre le matrici  $C$  e  $D$  mappano lo stato e gli ingressi diretti verso le prestazioni da ottimizzare ( $z$ ) e le variabili misurate e fornite in retroazione al controllore ( $y$ ).

Come mostrato in *Figura 3*, all'impianto  $P$  è 'agganciato' il controllore  $K(s)$ . Il sistema **closed-loop** (comprensivo delle matrici di peso, indicato con  $T_{zw}$ ) è ottenuto tramite la *Linear-Fractional-Transformation* (comando '**lft**' in MATLAB). Nel prossimo paragrafo vedremo come, tramite la sintesi  $H_\infty$ , si cercherà proprio di trovare il controllore che minimizza la *norma infinito*\*\* di tale sistema.

\*\*Si veda il Capitolo *Appendice, Norma Infinito - Test di bisezione*.



### 3.3 Controllore $H_\infty$

Una volta definita la formulazione in spazio di stato, la sintesi del controllore  $H_\infty$  procede secondo rigorosi step algebrici. Verificate le condizioni preliminari, l'algoritmo persegue l'obiettivo fondamentale di minimizzare la *norma infinito* del sistema in anello chiuso  $T_{zw}$ .

Fissato un valore di prova  $\gamma$ , il problema della sua ammissibilità viene ricondotto alla risoluzione di due Equazioni Algebriche di Riccati (*A.R.E.*). La prima determina la matrice definita positiva  $X_\infty$  per la retroazione ottima dello stato, la seconda la matrice definita positiva  $Y_\infty$  per l'osservatore. Affinché il valore di  $\gamma$  sia ammissibile, oltre alla positività delle soluzioni, è richiesto tassativamente che il raggio spettrale del loro prodotto sia strettamente limitato, ovvero  $\rho(X_\infty Y_\infty) < \gamma^2$ . Soddisfatto tale vincolo, l'algoritmo riduce progressivamente  $\gamma$  e reitera il calcolo.

Raggiunto il minimo, viene sintetizzato il *controllore centrale*, la cui architettura ricalca formalmente quella di uno stimatore con retroazione di stato, incapsulando in modo ottimo le specifiche di *loop-shaping*.

In MATLAB ciò è effettuato tramite il comando `hinfsyn`, che accetta in ingresso l'impianto generalizzato  $P$  ed il numero di ingressi misurati ed uscite di controllo del controllore.

#### MATLAB

```
1 .
2 %% Progetto del controllore H-Inf con prestazioni sul
   nominale
3 [K_inf, Tzw3, gamma, info] = hinfsyn(P, 2, 1);
4 K_inf = minreal(K_inf);
5 display(K_inf);
6
7 % sistema a ciclo chiuso (con le matrici di peso)
8 minreal(Tzw3);
9 format longG;
10 display(pole(Tzw3));
11 format short;
```

Il risultato è un controllore con 2 ingressi (misura di posizione e angolo) ed 1 uscita (tensione ai motori)

#### Command Window

```
>> size(K_inf)
State-space model with 1 outputs, 2 inputs, and 9 states.
```

e la seguente risposta in frequenza

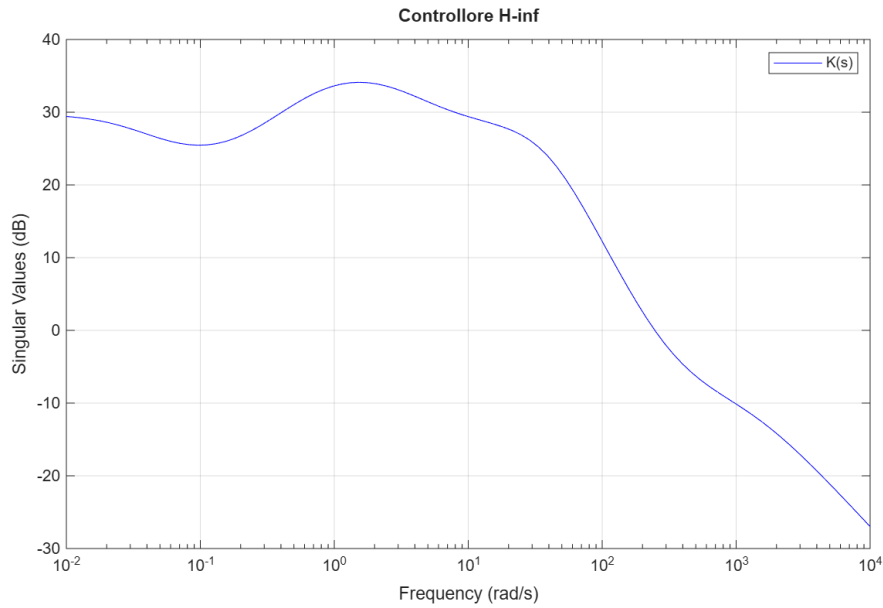


Figura 4: Risposta in frequenza  $K(s)$

L'andamento dei valori singolari di  $K(s)$  mostra un comportamento in frequenza coerente con gli obiettivi di *loop-shaping*. Si osserva un elevato guadagno alle basse frequenze, con un picco di circa 34 dB intorno a 1.5 rad/s, necessario per garantire un accurato inseguimento dei riferimenti e dominare le dinamiche dell'impianto. Ad alta frequenza, superati i 40 rad/s, si innesca un marcato *roll-off* che garantisce l'attenuazione del rumore di misura e previene la saturazione degli attuatori.

E' possibile vedere come tale controllore stabilizzi il sistema a ciclo chiuso.

#### Command Window

```
-1074.5 + 0i
-398 + 0i
-294.3 + 0i
-173.2 + 0i
-125.8 + 0i
-125.8 + 0i
-77.7 + 0i
-39.6 + 0i
-5.6 + 2.1i
-5.6 - 2.1i
-7.0 + 0i
-6.2 + 0i
-0.8 + 0.6i
-0.8 - 0.6i
-2.7 + 0i
-2.8 + 0i
-0.06 + 0i
-0.06 + 0i
```

### 3.4 Prestazioni sul nominale

In questo paragrafo vengono analizzate le prestazioni raggiunte sull'impianto nominale in seguito al controllo progettato.

**Nota:** non siamo più in fase di *design*, ma in fase di *analisi*. Si considera lo schema in feedback classico, senza le matrici di peso, e si pone come controllore proprio il  $K(s)$  progettato.

#### MATLAB

```
1 S_nom = minreal(feedback(eye(2), G0*K_inf));
2 T_nom = minreal(feedback(G0*K_inf, eye(2))); %closed-loop
   (G0)
3 KS_nom = minreal(K_inf*S_nom);
4 format short;
```

Le prestazioni ottenute sono le seguenti:

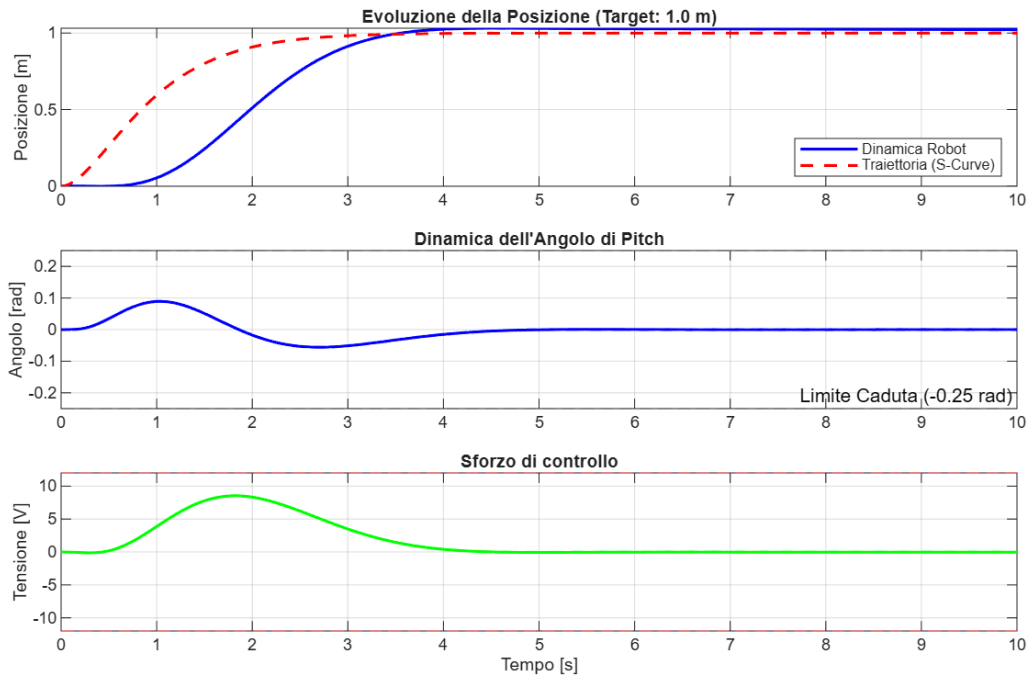


Figura 5: Validazione delle prestazioni nel dominio del tempo

La validazione del controllore  $H_\infty$  nel dominio del tempo è stata effettuata simulando l'inseguimento di una traiettoria di posizione a profilo di velocità smussato (S-Curve) con target a 1.0 m. L'analisi dei tre grafici conferma il perfetto bilanciamento tra reattività dinamica e rispetto dei vincoli fisici:

- **Evoluzione della Posizione (Sottoattuazione):** Il robot insegue il riferimento (curva tratteggiata rossa) raggiungendo il target di 1.0 m in circa 3.5

secondi. È corretto notare un ritardo iniziale tra il riferimento e la risposta del robot: a causa della dinamica a fase non minima (lo zero nel semipiano destro), il TWIP non può scattare in avanti istantaneamente, ma deve prima generare una coppia contraria per indurre lo sbilanciamento necessario al moto.

- **Dinamica dell'Angolo di Pitch:** Questo grafico illustra il principio di funzionamento del self-balancing robot. Per avviare il moto in avanti, il controllore comanda un'inclinazione positiva. Successivamente, per frenare e arrestarsi sul target, il robot deve "spingere" all'indietro, invertendo l'angolo di pitch. L'intera manovra è eseguita mantenendo l'inclinazione ampiamente all'interno della zona di sicurezza, lontana dal limite fisico di caduta ( $-0.25$  rad).
- **Sforzo di Controllo:** La tensione applicata ai motori DC evidenzia l'efficacia del peso  $W_u$  inserito in fase di sintesi. Il segnale di controllo è fluido e, soprattutto, si mantiene rigorosamente all'interno dei limiti fisici di alimentazione ( $\pm 12$  V), con un picco massimo di circa 8.5 V, evitando la saturazione degli attuatori.

E' stata analizzata in fase di simulazione anche la risposta ad un disturbo - un colpo di mano che inclina il robot di 0.05 rad - applicato quando questo è fermo ed in equilibrio

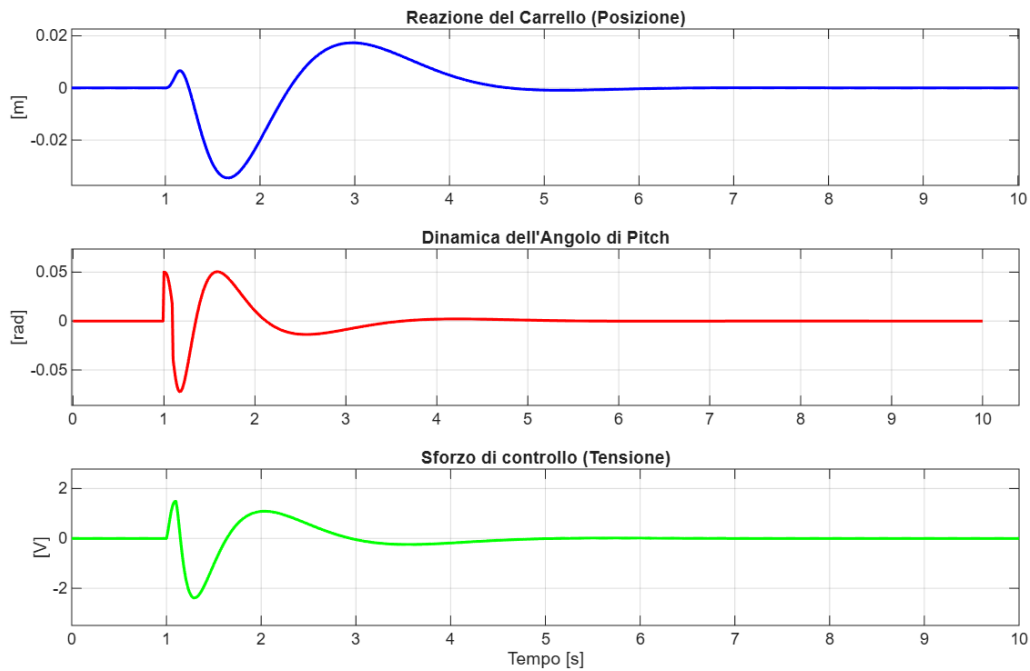


Figura 6: Validazione delle prestazioni nel dominio del tempo

In conclusione, la simulazione sul modello nominale dimostra che il controllore  $H_\infty$  risolve efficacemente il compromesso intrinseco del TWIP: garantisce un avanzamento fluido e preciso senza mai compromettere l'equilibrio della struttura.

## 4 Analisi della robustezza

Uno degli obiettivi primari del progetto è garantire robustezza a fronte delle variazioni parametriche definite nel Capitolo 2.1. Dopo aver verificato l'efficacia del controllore  $H_\infty$  nell'assicurare le specifiche richieste sull'impianto nominale, in questa sezione verrà verificata tale proprietà.

Definiamo innanzitutto le funzioni di sensitività sul sistema incerto

### MATLAB

```
1 S = minreal(feedback(eye(2), sys*K_inf));  
2 T = minreal(feedback(sys*K_inf, eye(2))); %closed-loop  
3 KS = minreal(K_inf*S);
```

### 4.1 Simulazione Monte-Carlo

Proviamo ad eseguire una simulazione come nel capitolo precedente, ma con le funzioni di sensitività appena definite

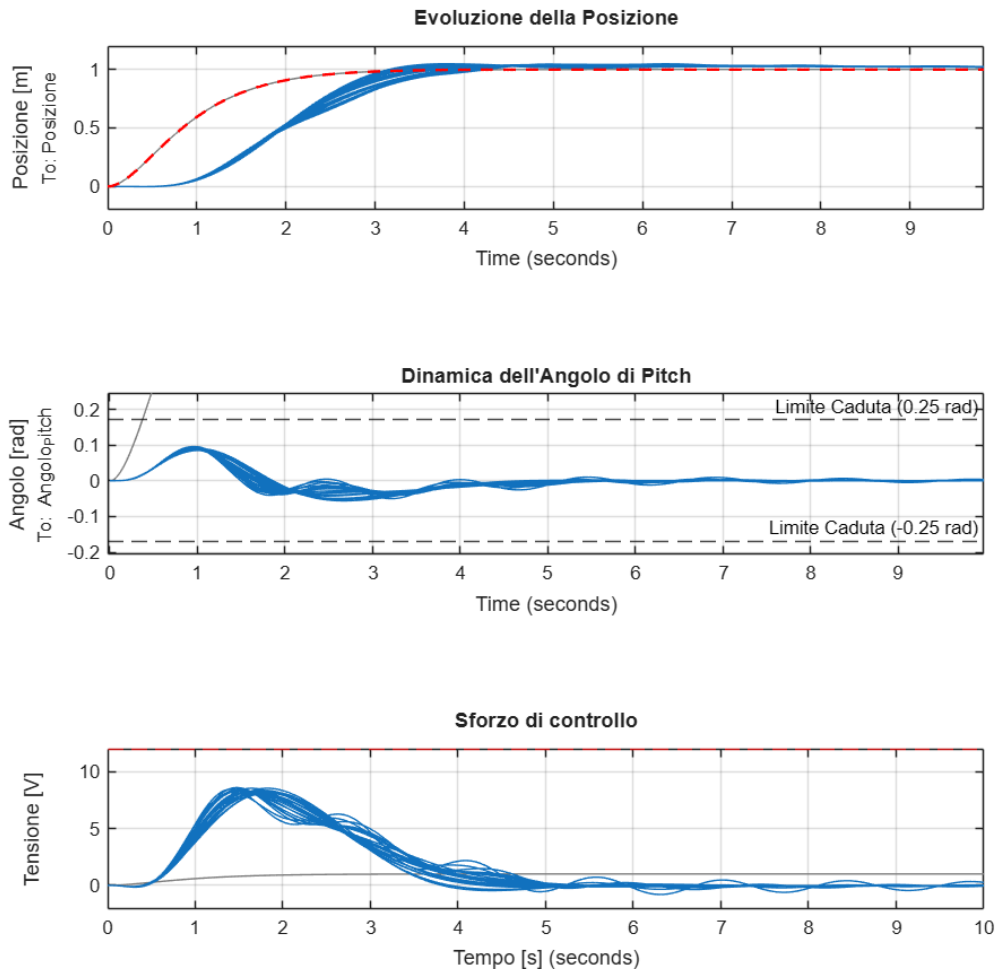


Figura 7: Sistema incerto

Sembrerebbe stabile. Tuttavia, ciò non è detto. La simulazione Monte-Carlo non è il modo corretto di analizzare l'incertezza: essendo una simulazione di  $n$  sistemi con combinazioni differenti dei parametri, potrebbe esserci una  $n+1$ -esima combinazione di questi che porta all'instabilità del sistema.

## 4.2 Dal Piccolo Guadagno all'Analisi $\mu$

Per valutare formalmente la stabilità robusta, la prassi metodologica prevede di riorganizzare il sistema perturbato a ciclo chiuso nella classica interconnessione  $M - \Delta$ : vengono separate le incertezze dal sistema, e modellate in una matrice  $\Delta$  diagonale, con  $\|\Delta\|_\infty \leq 1$ ; questo perchè, essendo entità fisiche ben distinte che entrano nell'impianto in punti specifici delle equazioni differenziali, non interagiscono tra loro.

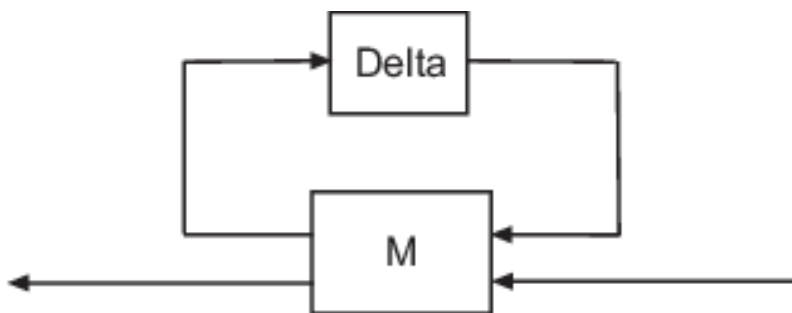


Figura 8: Struttura  $M - \Delta$  (fonte: mathworks.com)

Esplicitando i segnali in ingresso e in uscita:

$$\begin{bmatrix} y_\Delta \\ c \end{bmatrix} = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \begin{bmatrix} u_\Delta \\ r \end{bmatrix}$$

Il blocco  $M_{11}$  rappresenta la funzione di trasferimento "vista" esclusivamente dalle incertezze (è il canale cruciale per l'analisi della Stabilità Robusta), mentre  $M_{22}$  descrive la trasmissione nominale dal riferimento  $r$  all'uscita  $c$  (Prestazioni Nominali). Infine, i termini incrociati  $M_{12}$  e  $M_{21}$  quantificano l'accoppiamento tra la presenza dei disturbi parametrici e il degrado delle prestazioni.

In questo studio, è stato analizzato come primo approccio il Teorema del Piccolo Guadagno. Tale teorema stabilisce una condizione sufficiente molto potente: se la norma infinito della sottomatrice  $M_{11}$  risulta strettamente minore di 1, il sistema è garantito robustamente stabile per qualsiasi perturbazione ammissibile.

È cruciale sottolineare che un esito negativo non implica l'instabilità fisica del sistema perturbato, bensì espone l'eccessivo conservatorismo intrinseco del Teorema del Piccolo Guadagno. Questo criterio assume matematicamente che il blocco  $\Delta$  sia una matrice "piena" (non strutturata), valutando di conseguenza anche accoppiamenti incerti fisicamente impossibili. Al contrario, come definito nella modellazione del sistema, le perturbazioni fisiche del robot (variazioni di massa, tolleranze sull'inerzia e derivate elettriche) generano un blocco  $\Delta$  fortemente strutturato, configurabile come una matrice puramente diagonale.

## MATLAB

```
1 [M, Delta, BlkStruct] = lftdata(T); %separa M e Delta
```

Analizziamo il Delta

## Command Window

```
>> Delta
Uncertain matrix with 40 rows and 40 columns.
The uncertainty consists of the following blocks:
  Jp: Uncertain real, nominal = 0.004, variability =
      [-0.001,0.001], 9 occurrences
  K_mot: Uncertain real, nominal = 0.01, variability =
      [-10,10]%, 5 occurrences
  Mp: Uncertain real, nominal = 0.8, variability =
      [-1,50]%, 21 occurrences
  Rm: Uncertain real, nominal = 10, variability =
      [-15,15]%, 5 occurrences
```

Isoliamo e valutiamo la norma infinito di M11

## MATLAB

```
1 M11 = M(1:40, 1:40);
2 disp('Norma infinito di M11: ');
3 N_inf = norm(M11, 'inf');
4 disp(N_inf);
```

Esito:

## Command Window

```
N_inf =
    286.1587
```

Per ottenere una valutazione non conservativa, ricorriamo alla vera forma del  $\Delta$  strutturato. Si procederà tramite la  $\mu$  - *analisi*. Operativamente, ciò richiede la computazione del Valore Singolare Strutturato della sottomatrice  $M_{11}$  rispetto alla specifica topologia del blocco incerto  $\Delta$  (diagonale a blocchi).

A livello computazionale, il calcolo analitico esatto di  $\mu$  per matrici complesse risulta intrattabile, motivo per cui gli algoritmi di sintesi moderna implementano tecniche di ottimizzazione numerica mirate a individuare rigorosamente l'estremo superiore di tale valore al variare della frequenza, ovvero la *norma*  $\mu$ :

$$\|M_{11}\|_{\mu} = \sup_{\omega} \mu(M_{11}(j\omega)) \quad (9)$$

La condizione necessaria e sufficiente affinché l'impianto mantenga la stabilità robusta per qualsiasi combinazione ammissibile delle incertezze modellate è che:

$$\|M_{11}\|_{\mu} \leq 1 \quad (10)$$

#### MATLAB

```
1 [bounds, muinfo] = mussv(M11, BlkStruct);
2 disp('Norma-Mu di M11: ');
3 disp(max(max(squeeze(bounds.ResponseData))));
```

Analizzando i risultati

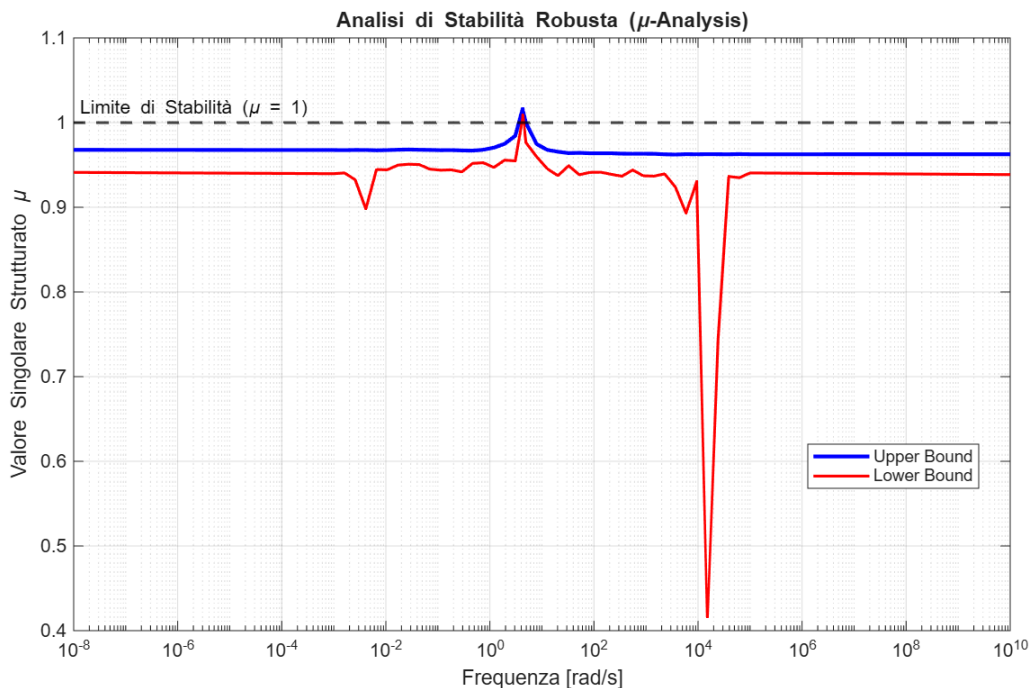


Figura 9: Upper e Lower Bounds di  $\mu$

#### Command Window

```
Norma-Mu di M11: 1.0164
```

Dal grafico e dal valore (approssimato) della  $\|M_{11}\|_{\mu}$ , si evince che non è garantita la stabilità robusta. Potrebbe verificarsi una combinazione dei parametri incerti che porta all'instabilità del sistema. Ottime prestazioni sono state raggiunte, ma la robustezza (requisito fondamentale) non è stata ottenuta.

Tramite il comando `robstab` di MATLAB è possibile analizzare in modo diretto se il sistema è robusto e, nel caso non lo fosse, qual è la combinazione dei parametri che lo destabilizza.



## MATLAB

```
1 opt = robOptions('Display', 'on', 'Sensitivity', 'on');  
2 [StabilityMargin, destabunc] = robstab(T, opt);
```

## Command Window

```
System is not robustly stable for the modeled uncertainty.  
-- It can tolerate up to 97.5% of the modeled uncertainty.  
-- There is a destabilizing perturbation amounting to 99%  
  of the modeled uncertainty.  
-- This perturbation causes an instability at the frequency  
  4.23 rad/seconds.  
-- Sensitivity with respect to each uncertain element is:  
4% for Jp. Increasing Jp by 25% decreases the margin by 1%.  
9% for K_mot. Increasing K_mot by 25% decreases the margin  
  by 2.25%.  
90% for Mp. Increasing Mp by 25% decreases the margin by  
  22.5%.  
0% for Rm. Increasing Rm by 25% decreases the margin by 0%.  
  
>> destabunc  
destabunc =  
struct with fields:  
Jp: 0.0050  
K_mot: 0.0110  
Mp: 1.1199  
Rm: 8.5146
```

Questo risultato orienta le successive scelte progettuali. Risulta infatti evidente come la ricerca di un hardware più performante si rivelerebbe una strategia del tutto inefficace, poiché non interverrebbe sulla reale vulnerabilità dell'impianto. L'unica soluzione consiste nell'operare un *detuning* del controllore. Rilassando le specifiche di inseguimento attraverso una calibrazione meno aggressiva delle matrici di peso, si renderà il regolatore meno sensibile alle variazioni della massa  $M_p$ .

E' nel capitolo successivo che verrà analizzato il *trade-off* che c'è tra robustezza e prestazioni del sistema.

## 5 Trade-off tra prestazioni e robustezza

Come evidenziato dall'analisi del Valore Singolare Strutturato, la sintesi iniziale focalizzata sull'ottimizzazione delle prestazioni nominali ha condotto a una violazione dei margini di stabilità robusta. Questo scenario rappresenta la manifestazione pratica di uno dei principi cardine della teoria dei controlli: il rigido *trade-off* (compromesso) tra reattività dinamica e tolleranza alle incertezze strutturali.

Un controllore troppo "teso" verso la prestazione perfetta sul modello ideale perde la flessibilità necessaria per assorbire un sovraccarico del 50% sulla massa del telaio o il calo fisiologico di coppia dovuto al surriscaldamento dei motori.

Per ricondurre il limite superiore di  $\mu$  al di sotto della soglia unitaria e garantire matematicamente la stabilità robusta in ogni scenario operativo, è necessaria una revisione del progetto di *loop-shaping*. Ciò richiede di allentare i vincoli imposti in fase di sintesi.

### 5.1 Modifica dei pesi

Le matrici di peso revisionate sono

| Peso        | Scelta                                         | Modifica                                         |
|-------------|------------------------------------------------|--------------------------------------------------|
| $W_{e,ang}$ | Filtro passa-basso, freq. di taglio: 16 rad/s  | Modifica del guadagno                            |
| $W_{e,pos}$ | Filtro passa-basso, freq. di taglio: 1.5 rad/s | Diminuzione della freq. di taglio e del guadagno |
| $W_u$       | Filtro passa-alto, freq. di taglio: 60 rad/s   | Modifica del guadagno                            |

#### MATLAB

```
1 We_ang = makeweight(5, 16, 0.5);  
2 We_pos = makeweight(25, 1.5, 0.1);  
3 Wu = makeweight(0.05, 60, 5);
```

Gli altri pesi sono rimasti invariati, seguendo le motivazioni effettuate nel capitolo 3.1

## 5.2 Sintesi del controllore riprogettato

Definiti i nuovi parametri, si è proceduto a una nuova sintesi ottima  $H_\infty$ . La figura seguente illustra il confronto in frequenza tra i valori singolari del controllore originario, focalizzato sulle prestazioni pure, e il nuovo regolatore.

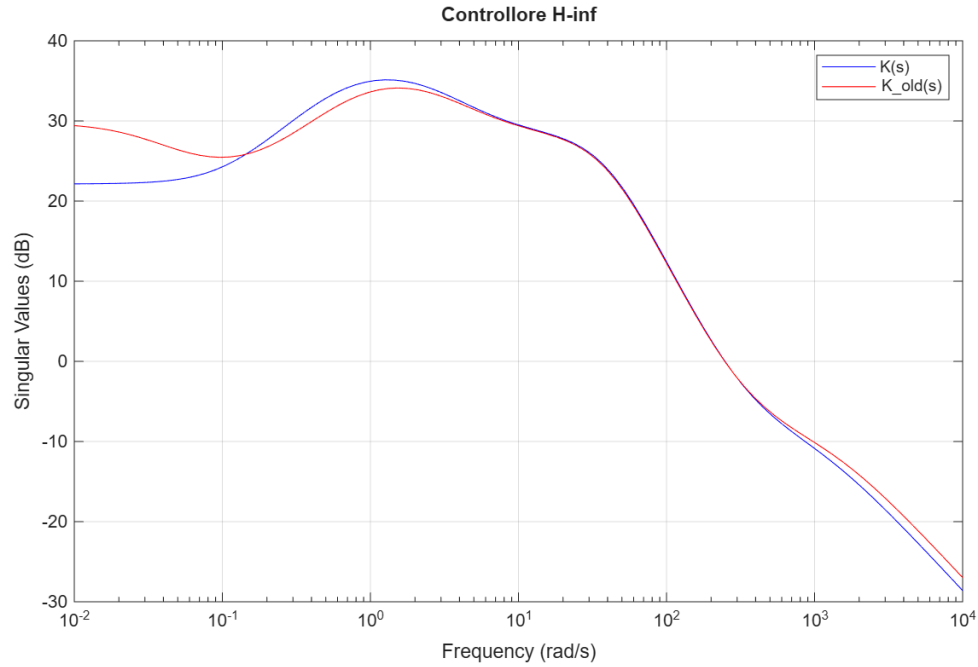


Figura 10: Confronto in frequenza (valori singolari) tra il controllore  $H_\infty$  originario ( $K_{old}$ , in rosso) e il controllore riprogettato ( $K$ , in blu).

### 5.3 Prestazioni ottenute

A seguito della modifica dell'anello di controllo, è stato verificato l'impatto sulle prestazioni nominali del sistema.

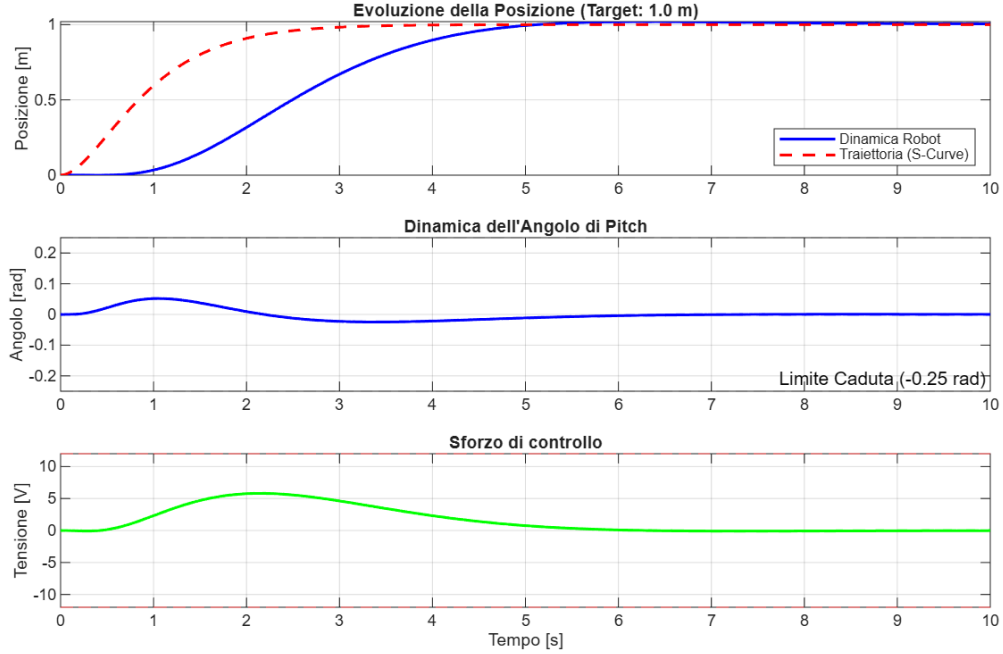


Figura 11: Risposta all'inseguimento di una traiettoria.

Parallelamente al *tracking*, è stata simulata nuovamente la reazione del TWIP all'applicazione di un disturbo.

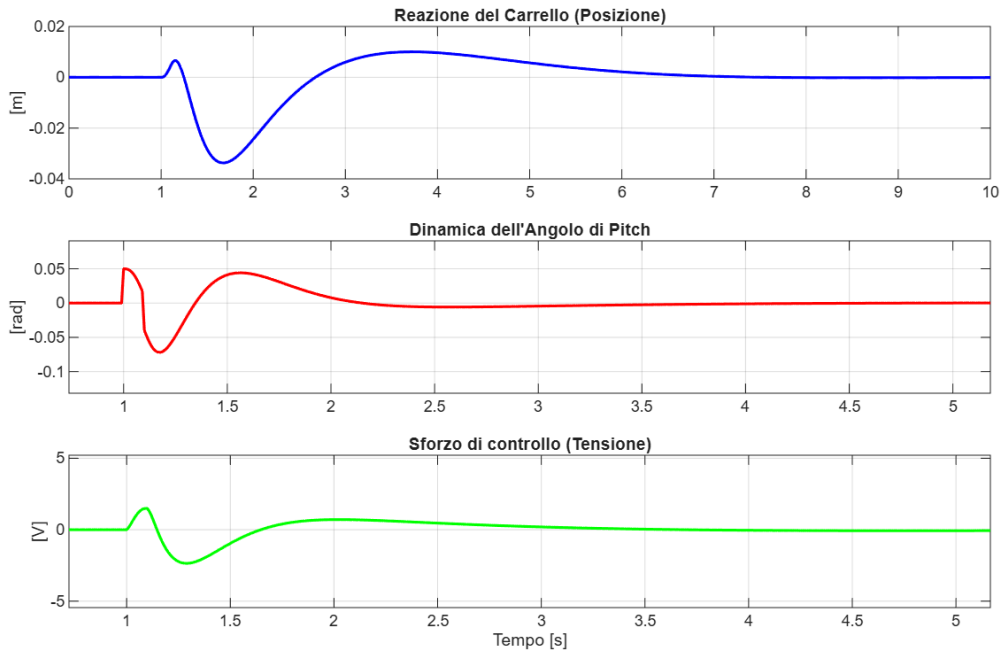


Figura 12: Risposta alla reiezione di un disturbo

## 5.4 Verifica della robustezza

L'ultima fase consiste nel verificare se il prezzo pagato abbia effettivamente garantito la stabilità robusta. Si procede pertanto all'estrazione formale della norma  $\mu$ .

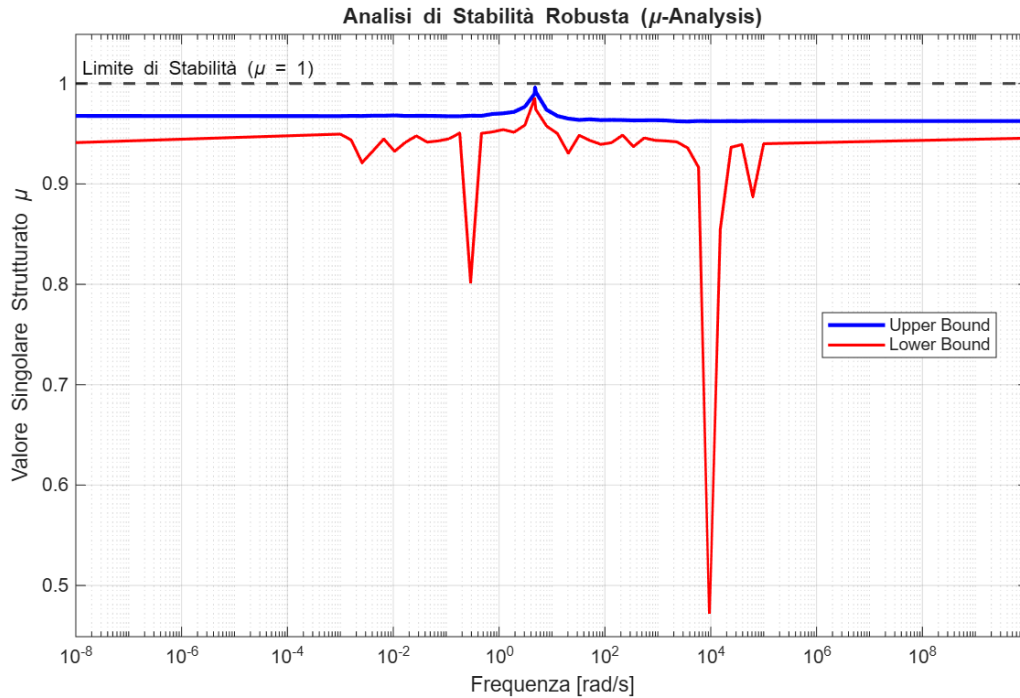


Figura 13: Nuovi Bounds del Valore Singolare Strutturato ( $\mu$ ).

### Command Window

```
Norma-Mu di M11:  
0.9959
```

```
Computing peak... Percent completed: 100/100  
System is robustly stable for the modeled uncertainty.  
-- It can tolerate up to 100% of the modeled uncertainty.  
-- There is a destabilizing perturbation amounting to 102%  
   of the modeled uncertainty.  
-- This perturbation causes an instability at the frequency  
   4.5 rad/seconds.
```

#### Command Window

```
-- Sensitivity with respect to each uncertain element is:  
1% for Jp. Increasing Jp by 25% decreases the margin by  
0.25%.  
4% for K_mot. Increasing K_mot by 25% decreases the margin  
by 1%.  
95% for Mp. Increasing Mp by 25% decreases the margin by  
23.8%.  
0% for Rm. Increasing Rm by 25% decreases the margin by 0%.
```

Il picco della norma  $\mu$ , attestandosi al valore di 0.9959, certifica in modo analitico e definitivo il raggiungimento della Stabilità Robusta per l'intero set di incertezze modellate.

In definitiva, per esplicitare numericamente l'entità del *trade-off* ingegneristico operato tra reattività e robustezza, la seguente tabella pone a confronto i principali indicatori di prestazione nominale tra l'architettura iniziale e l'architettura finale (riferiti all'inseguimento del gradino a 1.0 m).

| Indice di Prestazione                   | Controllore Iniziale | Controllore Robusto |
|-----------------------------------------|----------------------|---------------------|
| Tempo di assestamento ( $t_a$ )         | $\approx 3.5$ s      | $\approx 5.0$ s     |
| Picco angolare ( $\theta_{max}$ )       | 0.09 rad             | 0.05 rad            |
| Picco sforzo di controllo ( $V_{max}$ ) | 8.5 V                | 5.8 V               |

Tabella 2: Confronto delle prestazioni nominali.

## 6 Tentativi di Ottimizzazione

In questa fase del progetto, si è deciso di esplorare ulteriori margini di miglioramento architetturale sfruttando la parametrizzazione di Youla-Kucera. Come noto dalla teoria dei sistemi, l'insieme di tutti i controllori internamente stabilizzanti per un dato impianto può essere generato attraverso una trasformazione lineare frazionaria inferiore (LFT) tra un generatore "centrale"  $J(s)$  e un parametro dinamico, libero e asintoticamente stabile, denominato  $Q(s)$ . L'obiettivo è sfruttare i gradi di libertà offerti da  $Q$  per massimizzare le prestazioni e la robustezza.

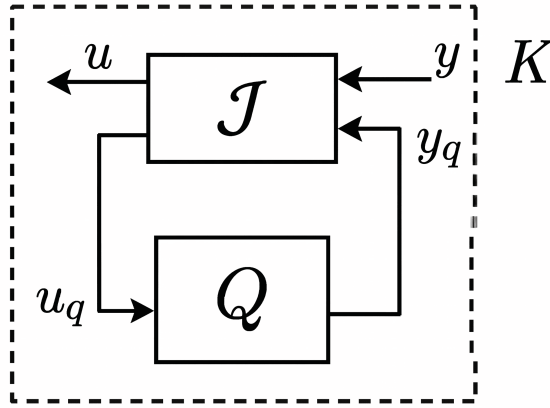


Figura 14: Controllore parametrizzato con Youla-Kucera.

### 6.1 Costruzione del generatore centrale $J$

Il primo step operativo è consistito nella costruzione del sistema aumentato  $J$ . Per garantire la stabilità nominale del blocco, le matrici di retroazione dello stato  $F$  e di iniezione dell'uscita  $L$  sono state calcolate risolvendo due problemi LQR duali per l'impianto nominale. Questo garantisce che la struttura di base del controllore mantenga il pendolo in equilibrio, delegando al parametro  $Q$  il compito di rifinire l'inseguimento e la reiezione dei disturbi.

$$J = \left[ \begin{array}{cc|cc} A + B_2F + LC_2 + LD_{22}F & \downarrow y & -L & B_2 + LD_{22} \\ u \rightarrow & F & 0 & I \\ u_q \rightarrow & -(C_2 + D_{22}F) & I & -D_{22} \end{array} \right]$$

## MATLAB

```
1 % Estrazione delle sottomatrici da P1
2 Ap = P1.A;
3 B2 = P1.B(:, 6); % Ingresso u di controllo
4 C2 = P1.C(6:7, :); % Uscite misurate y(1), y(2)
5 D22 = P1.D(6:7, 6); % Direct feedthrough
6
7 n = size(Ap, 1); ny = size(C2, 1); nu = size(B2, 2);
8 % Calcolo dei guadagni LQR per F e L
9 Q_f = eye(n); R_f = eye(nu);
10 F = -lqr(Ap, B2, Q_f, R_f);
11
12 Q_l = eye(n); R_l = eye(ny);
13 L = -lqr(Ap', C2', Q_l, R_l)';
14
15 % Costruzione della matrice J
16 A_J = Ap + B2*F + L*C2 + L*D22*F;
17 B_J = [-L, B2 + L*D22];
18 C_J = [F; -(C2 + D22*F)];
19 D_J = [zeros(nu, ny), eye(nu);
20        eye(ny), -D22];
21
22 J = minreal(ss(A_J, B_J, C_J, D_J));
```

## 6.2 Ottimizzazione Black-Box tramite systune

Invece di ricavare analiticamente la struttura di  $Q(s)$  interpolando i vincoli  $H_\infty$ , si è optato per un approccio di ottimizzazione numerica *black-box*. Tramite l'istruzione MATLAB `tunableSS`, il parametro incognito  $Q$  è stato inizializzato come un modello in spazio di stato generico e completamente sintonizzabile. L'ordine di  $Q$  è stato fissato a 6 stati, configurato con due ingressi (i segnali provenienti dalle uscite misurate) e un'uscita (l'azione di controllo).

La sintesi vera e propria è stata demandata all'algoritmo `systune`. L'obiettivo prestazionale è stato formalizzato attraverso l'oggetto `TuningGoal.Gain`, richiedendo che la norma infinito della matrice a ciclo chiuso  $T_{zw}$  fosse inferiore all'unità. Per scongiurare il confinamento dell'algoritmo in minimi locali, la ricerca è stata iterata utilizzando 30 punti di partenza pseudocasuali (`RandomStart`).



## MATLAB

```

1  % Definizione di Q come sistema sintonizzabile (Black-Box
   )
2  ordine_Q = 6;
3  Q_blackbox = tunableSS('Q_gen', ordine_Q, 1, 2);
4
5  % Costruzione del ciclo chiuso sintonizzabile
6  K_tunable = minreal(lft(J, Q_blackbox));
7  Tzw_tunable = lft(P1, K_tunable);
8  In_Tzw = {'d_in', 'd(1)', 'd(2)', 'n(1)', 'n(2)'};
9  Out_Tzw = {'ze(1)', 'ze(2)', 'zu', 'zc(1)', 'zc(2)'};
10
11 % Definizione vincolo (soft) H-inf e ottimizzazione
12 Req_Hinf = TuningGoal.Gain(In_Tzw, Out_Tzw, 1);
13 Options = systuneOptions('RandomStart', 30);
14 [Tzw_opt, fSoft, ~, Info] = systune(Tzw_tunable, Req_Hinf,
   , Options);
15
16 % Estrazione del controllore ottimizzato
17 Q_opt = getBlockValue(Tzw_opt, 'Q_gen');
18 K_opt = minreal(lft(J, Q_opt));

```

La convergenza dell'algoritmo ha permesso di estrarre il parametro ottimo  $Q_{opt}$  e di chiudere la LFT per ottenere il controllore finale  $K_{opt}$ .

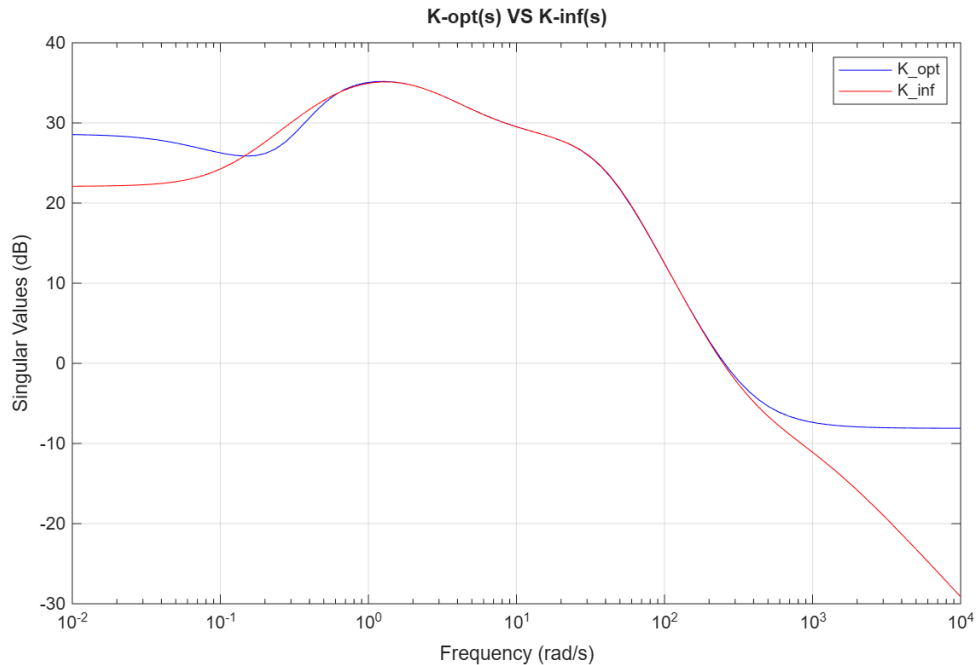


Figura 15: Confronto tra  $K_{opt}(s)$  ottenuto tramite parametrizzazione di Youla e  $K_{inf}(s)$  già ottenuto tramite sintesi  $H_\infty$ .

### 6.3 Validazione in Dominio del Tempo

La risposta nominale all'inseguimento della traiettoria (S-Curve con target a 1.0 m) mostra un *tracking* fluido, privo di sovraelongazioni eccessive.

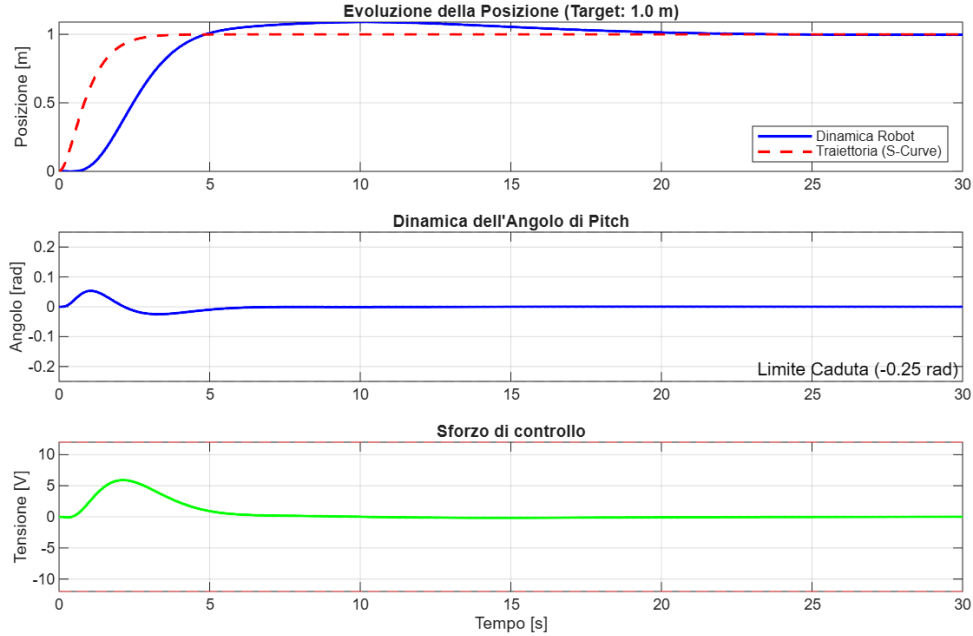


Figura 16: Tracking con il controllore  $K_{opt}$ .

E' interessante notare i risultati ottenuti sullo sforzo di controllo: vi è un picco massimo di circa 6 V, dunque il sistema gode di un ampio margine di sicurezza rispetto al limite fisico di saturazione degli attuatori ( $\pm 12$  V).

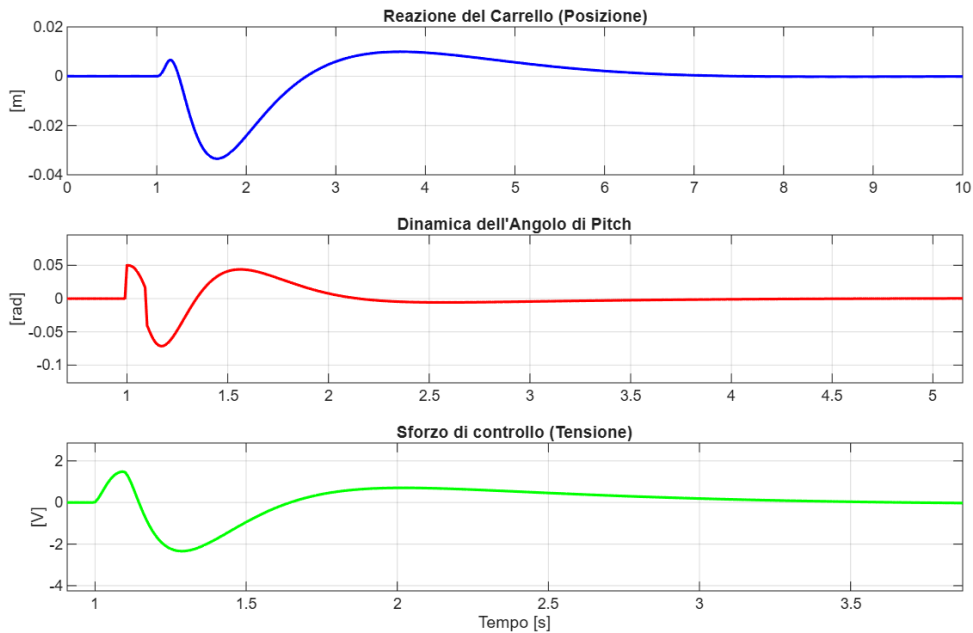


Figura 17: Risposta del sistema ad un disturbo

## 6.4 Robustezza del sistema

Dall'analisi della norma  $\mu$ , si evince come il sistema sia stabile robusto.

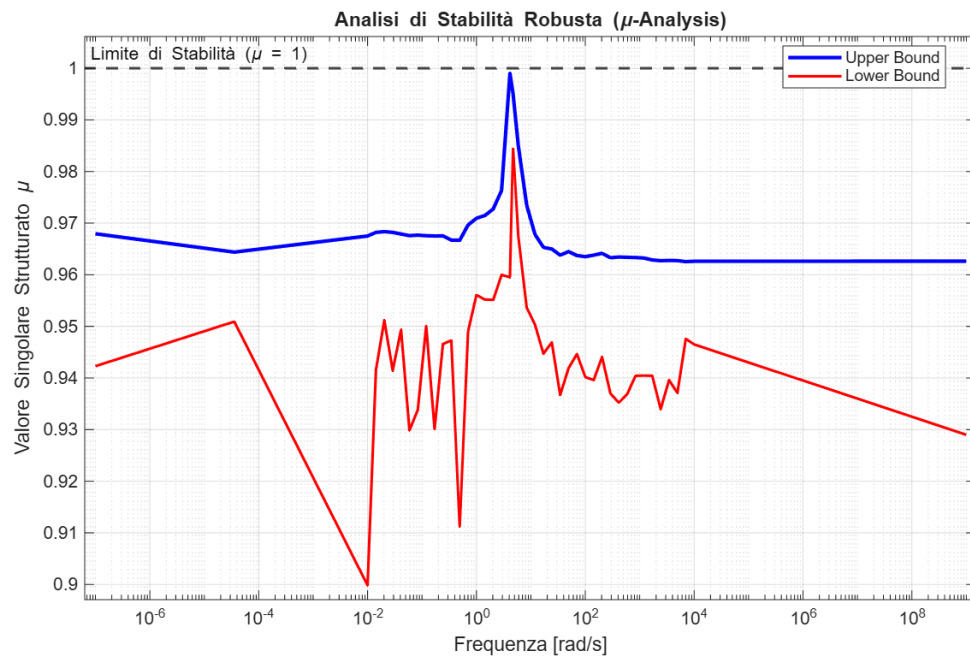


Figura 18: Risposta del sistema ad un disturbo

### Command Window

```
System is robustly stable for the modeled uncertainty.
-- It can tolerate up to 100% of the modeled uncertainty.
-- There is a destabilizing perturbation amounting to 102%
  of the modeled uncertainty.
-- This perturbation causes an instability at the frequency
  4.71 rad/seconds.
-- Sensitivity with respect to each uncertain element is:
1% for Jp. Increasing Jp by 25% decreases the margin by
0.25%.
5% for K_mot. Increasing K_mot by 25% decreases the margin
by 1.25%.
95% for Mp. Increasing Mp by 25% decreases the margin by
23.8%.
1% for Rm. Increasing Rm by 25% decreases the margin by
0.25%.
```

Con la seguente tabella sono messi a confronto i principali indicatori di prestazione nominale tra i due controllori (riferiti all'inseguimento del gradino a 1.0 m).

| <b>Indice di Prestazione</b>            | <b>Controllore <math>H_\infty</math></b> | <b>Controllore parametrizzato</b>                |
|-----------------------------------------|------------------------------------------|--------------------------------------------------|
| Tempo di assestamento ( $t_a$ )         | $\approx 5$ s                            | $\approx 5$ s, ma con maggiore margine di errore |
| Picco angolare ( $\theta_{max}$ )       | 0.05 rad                                 | 0.05 rad                                         |
| Picco sforzo di controllo ( $V_{max}$ ) | 5.8 V                                    | 5.8 V                                            |

## 6.5 Ottimizzazione del controllore centrale

La sintesi  $H_\infty$  standard non si limita a calcolare un singolo controllore, ma genera un'intera famiglia di controllori stabilizzanti sub-ottimi che garantiscono un livello di prestazione  $\|T_{zw}\|_\infty < \gamma$ . Questa famiglia di soluzioni è rigorosamente descritta dalla parametrizzazione di Youla-Kučera ed è esprimibile tramite una Trasformazione Lineare Frazionaria (LFT) tra un sistema generatore delle soluzioni, tipicamente denotato come  $K_{AS}$ , e un parametro libero  $Q(s)$ .

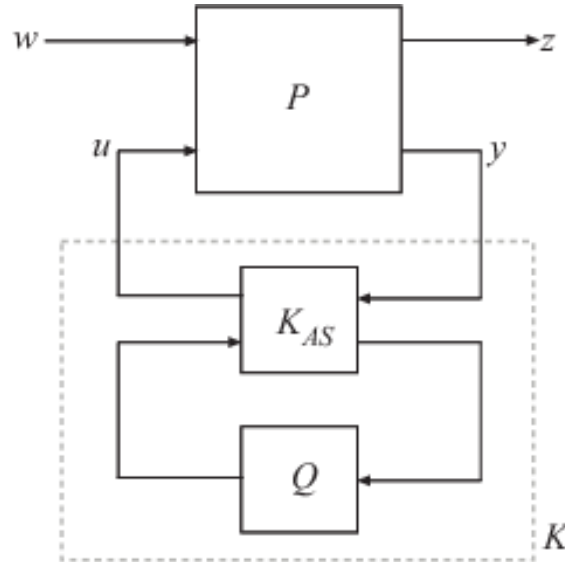


Figura 19: Parametrizzazione del controllore centrale (fonte: mathworks.com)

Affinché il sistema ad anello chiuso rimanga internamente stabile e rispetti i vincoli di robustezza imposti, la teoria stabilisce che il parametro di Youla  $Q(s)$  debba rispettare la condizione  $\|Q\|_\infty < \gamma$ .

Il controllore nominale restituito di default dagli algoritmi di sintesi basati sulle equazioni di Riccati (noto in letteratura come *controllore centrale*) si ottiene annullando il parametro libero, ovvero ponendo  $Q(s) = 0$ .

Al fine di indagare la possibilità di spingere le prestazioni del sistema oltre i limiti del controllore centrale, in questo lavoro di tesi si è implementata un'ottimizzazione algoritmica del parametro dinamico  $Q(s)$ . L'obiettivo era quello di sfruttare i gradi di libertà aggiuntivi forniti da  $Q(s)$  per minimizzare ulteriormente la norma della pianta generalizzata o migliorare il tracciamento temporale, pur rispettando i vincoli teorici della parametrizzazione.

Di seguito il codice MATLAB per il tentativo di ottimizzazione

#### MATLAB

```
1 K_AS = info.AS; % ottenuto dalla sintesi H-inf
2
3 ordine_Q = 3;
4 Q_hinf_tunable = tunableSS('Q_hinf', ordine_Q, 1, 2);
5
6 AP_in = AnalysisPoint('Q_in', 2); % 2 segnali che
    entrano in Q
7 AP_out = AnalysisPoint('Q_out', 1); % 1 segnale che esce
    da Q
8
9 Q_wrapped = AP_out * Q_hinf_tunable * AP_in;
10
11 K_tunable = minreal(lft(K_AS, Q_wrapped));
12 Tzw_tunable = lft(P1, K_tunable);
13
14 % obiettivo: norma_inf(Tzw) < 1 (soft - prestazioni)
15 In_Tzw = {'d_in', 'd(1)', 'd(2)', 'n(1)', 'n(2)'};
16 Out_Tzw = {'ze(1)', 'ze(2)', 'zu', 'zc(1)', 'zc(2)'};
17 Req_Hinf = TuningGoal.Gain(In_Tzw, Out_Tzw, 1);
18
19 % obiettivo: norma_inf(Q) < gamma (hard - stabilita')
20 Req_Q_norm = TuningGoal.Gain('Q_in', 'Q_out', gamma *
    0.99);
21 Req_Q_norm.Name = 'Vincolo_Norma_Q';
22
23 Options = systuneOptions('RandomStart', 30);
24
25 [Tzw_opt, fSoft, ~, Info] = systune(Tzw_tunable, Req_Hinf
    , Req_Q_norm, Options);
26
27 % estrazione del controllore ottimizzato
28 Q_opt_hinf = getBlockValue(Tzw_opt, 'Q_hinf');
29 K_opt_hinf = minreal(lft(K_AS, Q_opt_hinf));
```

Come si evince dalla *Figura 21*, i due controllori sono sostanzialmente identici.

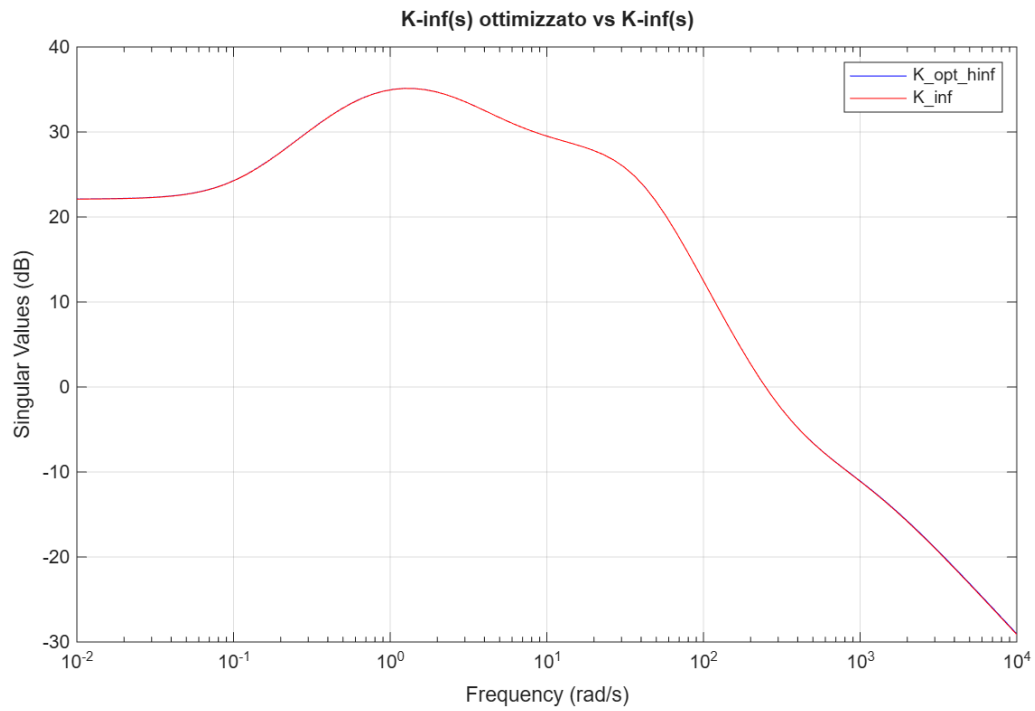


Figura 20: Confronto tra i controllori

L'analisi dei risultati ha dimostrato che la ricerca di un parametro  $Q(s) \neq 0$  non produce miglioramenti tangibili. Di conseguenza, anche le prestazioni sull'impianto nominale e la robustezza risultano le stesse.

## 7 Riduzione dell'ordine del controllore

I controllori sintetizzati attraverso le tecniche di controllo robusto  $H_\infty$  presentano intrinsecamente un ordine dinamico elevato, tipicamente pari alla somma degli stati della pianta fisica e di tutti i filtri di peso associati alla pianta generalizzata. Al fine di rendere il controllore compatibile con i limiti computazionali di un microcontrollore reale e ridurre il costo di implementazione, si rende necessaria una procedura algoritmica di riduzione dell'ordine, operando in modo da non degradare la robustezza e le prestazioni ad anello chiuso.

In questo lavoro, la procedura è stata condotta adottando l'approccio del troncamento bilanciato. In una prima fase diagnostica, il controllore ottimizzato è stato trasformato nella sua realizzazione bilanciata, in cui i gradi di osservabilità e controllabilità di ciascuno stato coincidono. Lo studio dei Valori Singolari di Hankel ha permesso di quantificare il contributo energetico esatto di ogni singola variabile di stato alla dinamica complessiva ingresso-uscita. L'analisi visiva di tale spettro energetico ha evidenziato un decadimento oltre i primi 5 stati dominanti.

A valle della sintesi, la validità dell'approssimazione è stata rigorosamente accertata verificando la stabilità interna del nuovo sistema ad anello chiuso, rendendo il controllore ridotto pronto per l'implementazione sul prototipo fisico.

### MATLAB

```
1 R = reducespec(K_inf, "balanced");
2 figure('Name', 'Analisi delle Energie degli Stati (
    reducespec)');
3 % grafico dei s.v. di Hankel, con linea d'errore massimo
4 view(R);
5 % fa tutto in automatico
6 K_ridotto = getrom(R, 'Order', 5);
7
8 %% Alternativa 'manuale':
9 stati_da_mantenere = 5;
10 % K in rapp. bilanciata
11 [K_bal, ~] = balreal(K_inf);
12 stati_da_eliminare = (stati_da_mantenere + 1):order(K_bal
    );
13 % eliminazione degli stati
14 K_ridotto = xelim(K_bal, stati_da_eliminare);
```



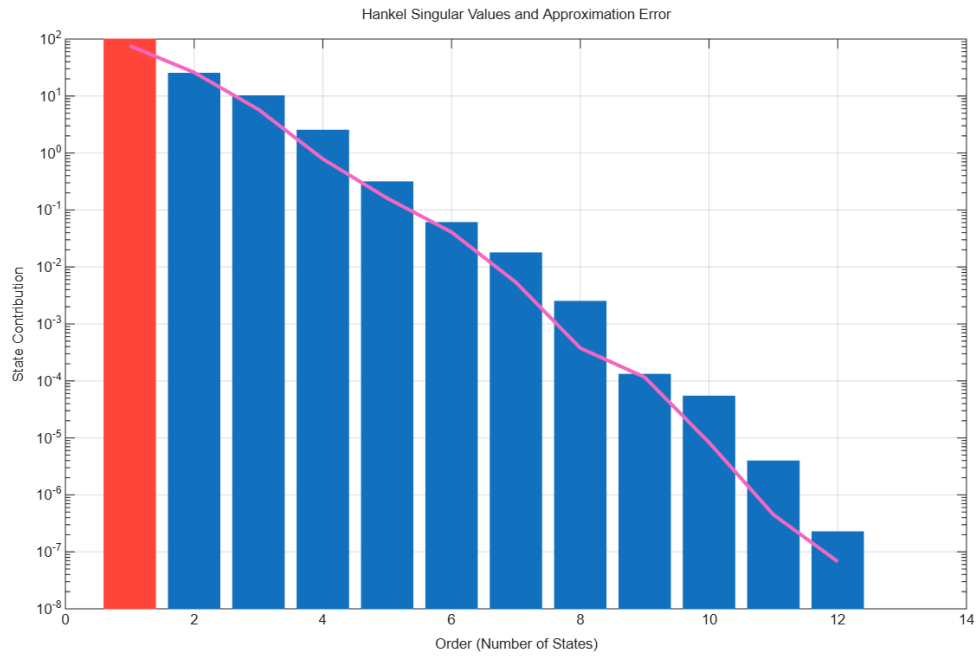


Figura 21: Valori Singolari di Hankel

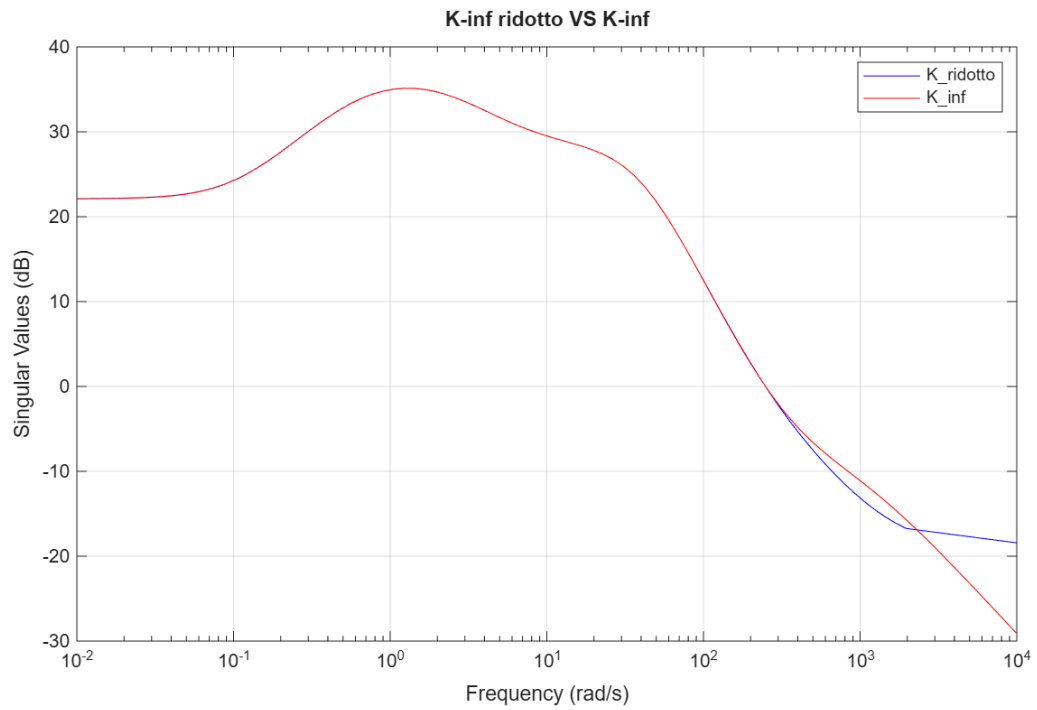


Figura 22: Confronto post-riduzione

## 8 Appendice

### 8.1 Norma infinito - Test di bisezione

La *norma infinito* di un sistema è definita come l'estremo superiore del massimo valore singolare della matrice di trasferimento al variare di  $\omega$ , ovvero

$$\|G\|_{\infty} = \sup_{\omega} \bar{\sigma}(G(j\omega)) \quad (11)$$

Il suo calcolo non viene effettuato in modo analitico, ma tramite un vero e proprio *test di bisezione*. Di seguito è stato implementato l'algoritmo in MATLAB.

#### MATLAB

```
1 function gamma_opt = norma_inf(sys)
2     % input: sys -> sistema LTI (modello ss)
3     gamma_min = 0;
4     gamma_max = 50;
5     % tolleranza dell'algoritmo di bisezione
6     tol_bisezione = 1e-3;
7     A = sys.A; B = sys.B; C = sys.C; D = sys.D;
8     % Se gamma_max iniziale troppo piccolo, lo alziamo.
9     while testGammaRiccati(A, B, C, D, gamma_max)==false
10         gamma_max = gamma_max * 5;
11         if(gamma_max>1000000)
12             gamma_opt = 'Inf';
13             return;
14         end
15     end
16     % Bisezione
17     iter = 0;
18     while (gamma_max - gamma_min) > tol_bisezione
19         iter = iter + 1;
20         gamma_mid = (gamma_min + gamma_max) / 2;
21
22         if testGammaRiccati(A, B, C, D, gamma_mid)
23             % gamma_mid tetto valido, abbassiamo
24             gamma_max = gamma_mid;
25         else
26             % gamma_mid troppo basso, alziamo
27             gamma_min = gamma_mid;
28         end
29     end
30     gamma_opt = gamma_max;
31 end
```

## MATLAB

```

1 function is_valid = testGammaRiccati(A, B, C, D, gamma)
2     % costruisce la matrice Hamiltoniana e la testa
3     I = eye(size(D, 2));
4     R = (gamma^2) * I - (D' * D);
5     % R deve essere definita positiva. E' il primo step:
6     % se non verificata, non
7     % va avanti, ma alza direttamente gamma.
8     if min(eig(R)) <= 0
9         is_valid = false;
10        return;
11    end
12    % Costruzione della matrice hamiltoniana
13    invR = R \ eye(size(R));
14    H11 = A + B * invR * D' * C;
15    H12 = B * invR * B';
16    H21 = -C' * (eye(size(C,1)) + D * invR * D') * C;
17    H22 = -H11';
18
19    H_gamma = [H11, H12; H21, H22];
20
21    % Verifica tramite la logica di Riccati
22    is_valid = checkRiccatiDomain(H_gamma);
23 end
24
25 function is_riccati = checkRiccatiDomain(H)
26     % verifica se la matrice H appartiene a Ric
27     n = size(H, 1) / 2;
28     tol = 1e-9; %usata una tolleranza numerica
29
30     [W, D] = eig(H); % calcolo autovalori e autovettori
31     autovalori = diag(D);
32     %1. H non deve avere autovalori sull'asse immaginario
33     if any(abs(real(autovalori)) < tol)
34         is_riccati = false;
35         return;
36     end
37     %2. V1 deve essere invertibile
38     % autovalori stabili (Re()<0)
39     indici_stabili = real(autovalori) < -tol;
40     % base del sottospazio stabile e prendiamo V1
41     V_stabile = W(:, indici_stabili);
42     V1 = V_stabile(1:n, :);
43     % controllo se V1 invertibile
44     if rank(V1) < n
45         is_riccati = false;
46         return;
47     end
48     is_riccati = true;
49 end

```

Verifica con il comando `norm` di MATLAB con il sistema a ciclo chiuso T-nom con controllore K-inf.

**Command Window**

```
>> norm(T_nom, 'inf')
ans =
    7.5372

>> norma_inf(T_nom)
ans =
    7.5378
```